

main函数种类

main函数标准类型

- 无参:

```
1 | int main(void) { return 0; }
```

- 有参:

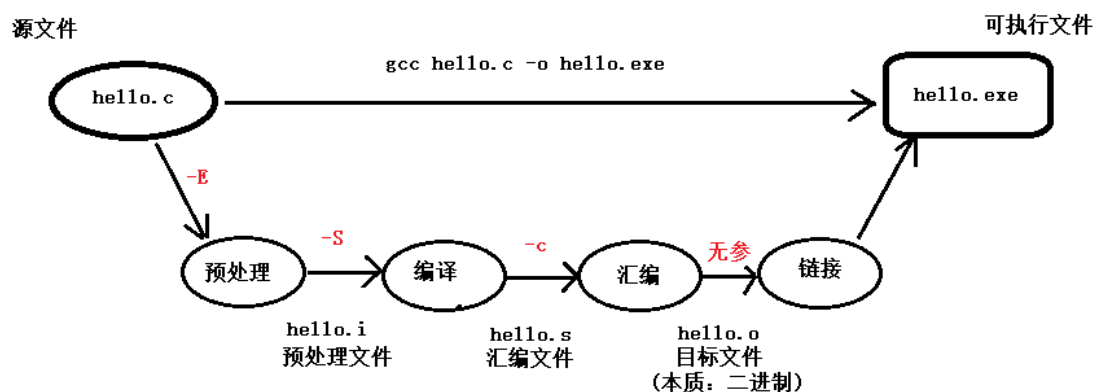
```
1 | int main(int argc, char *argv[]) { return 0; }
```

main函数其他类型

```
1 | // 都能正常运行。但是！不是main的标准语法格式。  
2 | void main(int argc, char *argv[]);  
3 | void main(void);  
4 | int main();  
5 | int main(void);  
6 | main();  
7 | main(int argc, char *argv[]);
```

gcc编译4步骤

整体过程



预处理

- 参数: -E
- 生成的文件: xxx.i 预处理文件
- 使用命令: gcc -E xxx.c -o xxx.i
- 工具: 预处理器 (包含在 gcc 编译集合工具中)

- 完成的工作：

1. 头文件展开。

- 展开 stdio.h 文件内容，和源码一起，放到 xxx.i 文件中。
- **不检查语法错误！** 可以在此阶段展开任意文件。
- 测试命令：gcc -E hello.c -o hello.i -I(大i) .(当前目录)

2. 宏定义替换

- 将宏名，替换成宏值。
- #define PI 3.14 【解释】：define：创建宏。PI：宏名 3.14：宏值。

```
1 // 测试案例。 使用命令： gcc -E hello.c -o hello.i
2 #include <stdio.h>
3
4 // 定义宏
5 #define PI 3.14
6
7 int main(void)
8 {
9     printf("hello world\n");
10
11     // 使用宏
12     printf("PI = %f\n", PI);
13
14     return 0;
15 }
```

3. 替换注释

- 把单行、多行注释替换成空行。

4. 展开条件编译

- 根据条件来展开代码。

```
1 // 测试案例
2 #include <stdio.h>
3
4 // 定义宏
5 // #define PI 3.14 // PI 定义与否，直接决定了下面的 ----666 是否打印。
6
7 int main(void)
8 {
9     printf("hello world\n");
10
11     // 使用宏
12     //printf("PI = %f\n", PI);
13
14     // 使用条件编译，含义是：如果定义了PI，那么就打印 -----666， 否则不打印。
15     #ifdef PI
16         printf("-----666\n");
17     #endif
18     return 0;
19 }
```

- 参数: -S
- 生成的文件: xxx.s 汇编文件
- 使用命令: gcc -S xxx.i -o xxx.s
- 工具: 编译器 (包含在 gcc 编译集合工具中)
- 完成的工作:
 1. 逐行检查语法错误! 【重点】—— 编译过程整个gcc编译4步骤中, 最耗时。
 2. 将 C 程序翻译成汇编指令。得到 .s 汇编文件。

```

1      .file    "hello.c"
2      .text
3      .def     __main; .sc1    2; .type   32; .endif
4      .section .rdata,"dr"
5      .LC0:
6      .ascii   "hello world\0"
7      .LC2:
8      .ascii   "PI = %f\12\0"
9      .LC3:
10     .ascii   "-----6666\0"
11     .text
12     .globl   main
13     .def     main; .sc1    2; .type   32; .endif
14     .seh_proc main
15 main:
16     pushq   %rbp
17     .seh_pushreg %rbp
18     movq    %rsp, %rbp
19     .seh_setframe %rbp, 0
20     subq    $32, %rsp
21     .seh_stackalloc 32
22     .seh_endprologue
23     call    __main
24     leaq    .LC0(%rip), %rcx
25     call    puts
26     movq    .LC1(%rip), %rax
27     movq    %rax, %rdx
28     movq    %rdx, %xmm1
29     movq    %rax, %rdx
30     leaq    .LC2(%rip), %rcx
31     call    printf
32     leaq    .LC3(%rip), %rcx
33     call    puts
34     movl    $0, %eax
35     addq    $32, %rsp
36     popq    %rbp
37     ret
38     .seh_endproc
39     .section .rdata,"dr"
40     .align   8
41     .LC1:
42     .long    1374389535
43     .long    1074339512
44     .ident   "GCC: (x86_64-posix-sjlj-rev0, Built by MinGW-w64 project)
45     .def     puts; .sc1    2; .type   32; .endif
46     .def     printf; .sc1    2; .type   32; .endif

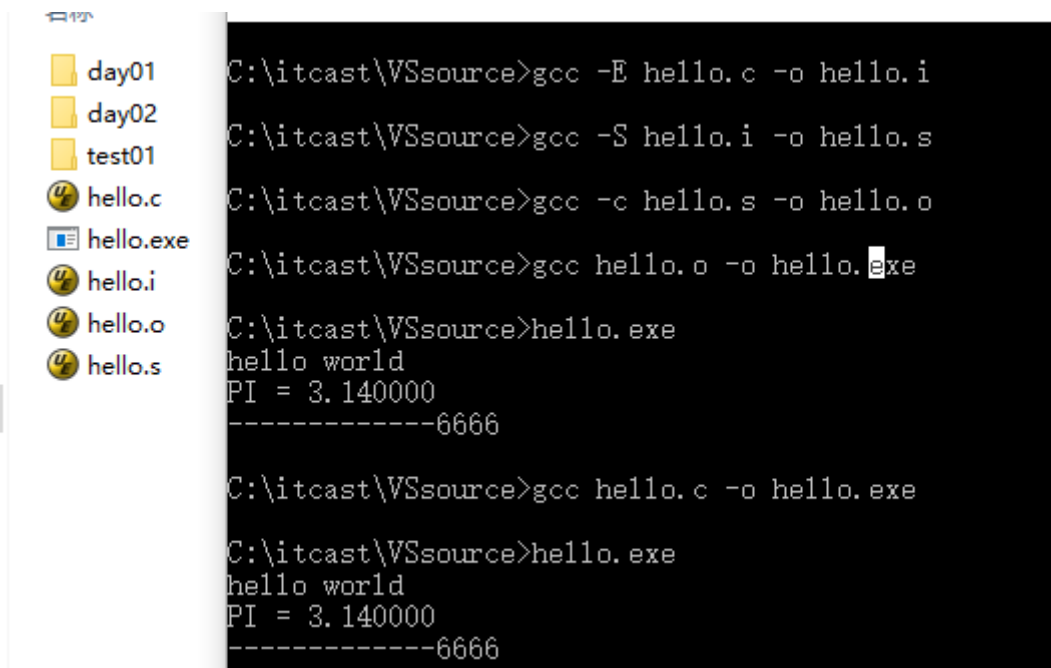
```

汇编

- 参数: -c
- 生成的文件: xxx.o 目标文件 (二进制。人类看不懂。)
- 使用命令: gcc -c xxx.s -o xxx.o
- 工具: 汇编器 (包含在 gcc 编译集合工具中)
- 完成的工作:
 - 翻译: 将汇编指令翻译成对应的 二进制指令。

连接

- 参数: 无 (-o 不是链接阶段参数, 是用来指定文件名)
- 生成的文件: xxx.exe 可执行文件 (二进制。人类看不懂。)
- 使用命令: gcc xxx.o -o xxx.exe
- 工具: 链接器 (包含在 gcc 编译集合工具中)
- 完成的工作:
 - 库引入
 - 合并多目标文件
 - 合并启动例程。

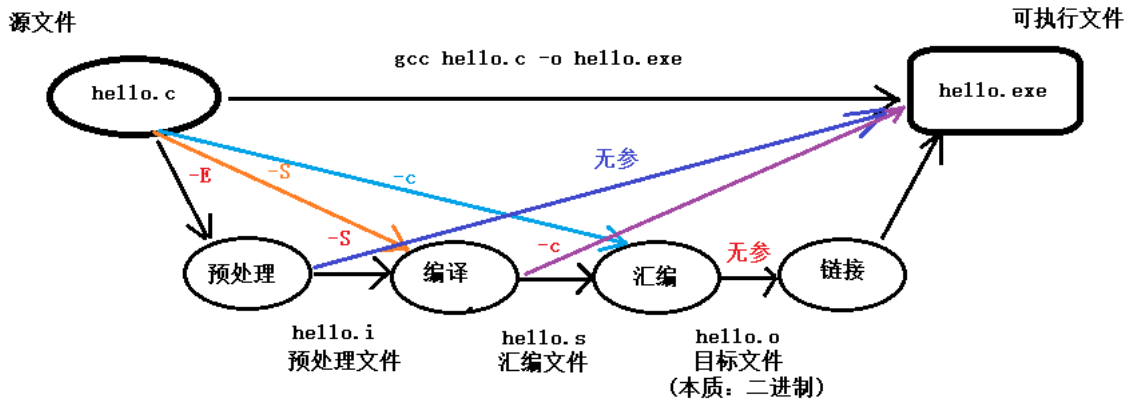


```
C:\itcast\VSsource>gcc -E hello.c -o hello.i
C:\itcast\VSsource>gcc -S hello.i -o hello.s
C:\itcast\VSsource>gcc -c hello.s -o hello.o
C:\itcast\VSsource>gcc hello.o -o hello.exe
C:\itcast\VSsource>hello.exe
hello world
PI = 3.140000
-----6666

C:\itcast\VSsource>gcc hello.c -o hello.exe
C:\itcast\VSsource>hello.exe
hello world
PI = 3.140000
-----6666
```

小结

gcc 编译的 4 个步骤中, 每个步骤直接都可相互转换。



printf格式化输出int

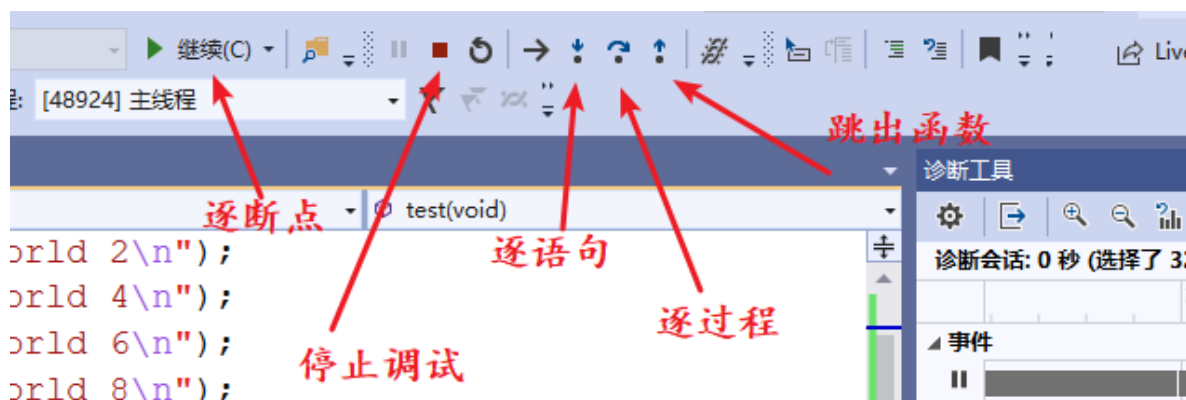
```
1 #include <stdio.h>
2
3 #define PI 3.14
4
5 int main(void)
6 {
7     int a = 10;
8
9     printf("%d\n", a);      // %d: 格式匹配符，匹配整数。
10    printf("a = %d\n", a);  // a = 在""中，代表普通字符串，原样输出。
11    printf("%d\n", 100);
12
13    printf("%f\n", PI);     // %f: 格式匹配符，匹配小数。
14    printf("PI = %f\n", PI); // PI = 在""中，代表普通字符串，原样输出。
15
16    printf("%f\n", 3.45678); // %f: 格式匹配符，匹配小数。
17
18    int b = 20;
19
20    printf("%d + %d = %d\n", a, b, a+b); // +, = 在""中，代表普通字符串，原样输出。
21    printf("%d + %d = %d\n", 3, 7, 3+7);
22    return 0;
23 }
```

程序调试

- 前提：程序，没有语法错误。——语法错误 VS 帮我检查。
 - 检查 程序出现的逻辑错误！
- 核心思想：
 - 让程序一行一行的执行。
- 添加行号：
 - 工具 —— 选项 —— 文本编辑器 —— c/c++ —— 行号（勾选）

程序调试流程

1. 添加断点。—— 可以添加多个
 1. 鼠标点击待添加断点行，左侧行号前灰色区域。再次点击取消。
 2. 光标停止在待添加断点行的任意位置，按F9添加断点。再次按 F9 取消断点。
2. 调试，必须在 Debug 模式下进程。Release 模式无效。
3. F5 启动调试。
4. 断点停止的位置，是尚未执行的指令。
5. 开始调试：
 1. 逐语句执行。逐语句执行下一行 (F11)：遇见函数，进入自定义函数内部，逐条跟踪执行。
 2. 逐过程执行。逐过程执行下一行 (F10)：遇见函数，不进入函数内部，逐条跟踪执行。
 3. 逐断点执行。代码中有多断点，直接跳转到下一个断点。—— 点击“继续”，无快捷键。
 4. 跳出函数。跳出当前断点所在的函数。shift + F10

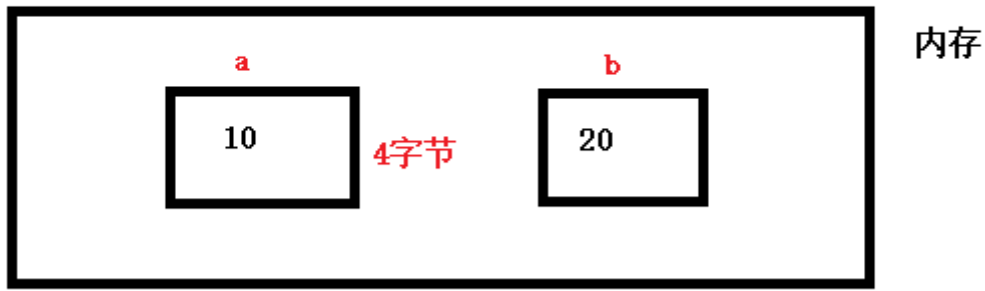


变量

变量 3 要素

1. 变量名：用来在程序中使用。
2. 变量类型：开辟内存空间大小。
3. 变量值：存储的实际数据。

a、b 变量名
空间大小 变量类型
数值10 变量值 变量 3 要素。
 int a = 10



变量定义：

- 定义语法：类型名 变量名 = 变量值（一般定义方法）。int m = 57;
- 会开辟内存空间给变量。变量声明不会开辟内存空间。

变量声明：

- 语法：
 - int a; 没有变量值的变量定义，叫做声明。
 - extern int a; 添加 extern 关键字。
- 特性：
 - 变量要想使用，**必须** 有定义。
 - 编译器，在使用变量之前，必须要看到变量定义。如果没有看到，编译器会自动找寻一个变量声明。提升成定义。
 - 如果 变量声明前，添加了 extern 关键字，无法提升！

常量

不会变化、不能被修改的数据。

1. "hello"、'A'、57、-10、3.1415926（浮点常量）
2. #define PI 3.14 ——【宏定义】语法：#define 宏名 宏值
 - 强调：没有 ";" 结束标记。
 - 强调：没有 "="
3. const int a = 10; 定义语法：const 类型名 变量名 = 变量值
 - const 关键字：被该关键字修饰的变量，表示为只读变量。

练习：

```
1 #define PI 3.1415926           // 定义常量
2
3 int main(void)
4 {
5     // 圆的面积 : s = PI * r * r
6     // 圆的周长 : L = 2 * PI * r
7     int r = 3;                  // 变量的定义。
```

```

8
9     float s = PI * r * r;    // 表达式。作为变量值。
10    float l = 2 * PI * r;
11
12    //printf("圆的面积: %f\n", s); // 28.274334 默认显示 6 位小数。
13    //printf("圆的周长: %f\n", l); // 18.849556
14
15    //printf("圆的面积: %.2f\n", s); // 28.27
16    //printf("圆的周长: %.2f\n", l); // 18.85指定保留小数点后两位，对第3位四舍五入
17
18    float m = 3.4567891;
19
20    printf("m=%.2f\n", m);
21    //共显示6位数，包含小数点，保留小数点后两位，对第3位四舍五入，不足6位用空格补齐。
22
23    printf("m=%06.2f\n", m);
24    //共显示6位数，包含小数点，保留小数点后两位，对第3位四舍五入，不足6位用0补齐。
25
26    return 0;
27 }

```

标识符

- 变量和常量总称

硬性要求

1. 标识符不能是关键字、函数名。system、printf、int、main、return。。。
2. 只能有 字母、数字、下划线组成。a-z/A-Z/0-9_
 - abc_1/ abc_2 / _abc_1 / a_b_c_d
3. 不能以数字开头。
 - int a5ir = 10; ok
 - int _34F = 6; ok
 - float 98ti_54 = 5.4; error;
4. 大小写严格区分。
 - 通常 使用大写来定义常量。#define MAX 100
 - 通常 使用小写来定义变量。

命名规范

1. 大驼峰法: int HelloWorldHahaHohoHehe = 10;
 - 多个单词组成变量名，每个单词首字母大写。
2. 小驼峰法: int helloWorldHahaHohoHehe = 10;
 - 多个单词组成变量名，首个单词的首字母小写，其余每个单词首字母大写。
3. 小写+下划线: int hello_world_haha_hoho_hehe = 10;
 - C 语言专用!

sizeof关键字

- sizeof 不是函数。
- 求取变量、数据类型，占用的内存空间大小。
- 使用方法：
 1. sizeof(变量名) —— 返回变量大小，单位整数字节。
 2. sizeof(类型名) —— 返回 数据类型大小，单位整数字节。
 3. sizeof 变量名 —— 语法C语言支持该写法，不推荐使用。

整型

有符号整型

- 获取数据类型的最小值，最大值，可以使用 #include <limits.h>

整型名	名称	格式匹配符	占用的大小	最小值	最大值
int	整型	%d	4字节	-2147483648	2147483647
short	短整型	%hd	2字节	-32768	32767
long	长整型	%ld	windows: 32/64位: 4字节 Linux下: 32位: 4字节、64位: 8字节。	-2147483648	2147483647
long long	长长整型	%lld	8字节	-9223372036854775808	9223372036854775807

```
1  #include <stdio.h>
2  #include <limits.h>
3
4  int main(void)
5  {
6      printf("int大小 = %u\n", sizeof(int));
7      printf("int最小值: %d, 最大值: %d\n", INT_MIN, INT_MAX);
8
9      printf("short大小 = %u\n", sizeof(short));
10     printf("short最小值: %hd, 最大值: %hd\n", SHRT_MIN, SHRT_MAX);
11
12     printf("long大小 = %u\n", sizeof(long));
13     printf("long最小值: %ld, 最大值: %ld\n", LONG_MIN, LONG_MAX);
14
15     printf("long long大小 = %u\n", sizeof(long long));
16     printf("long long最小值: %lld, 最大值: %lld\n", LLONG_MIN, LLONG_MAX);
```

```
17  
18     return 0;  
19 }
```

无符号整型

整型名	名称	格式 匹配 符	占用的大小	最 小 值	最 大 值
unsigned int	无符号 整型	%u	4字节	0	
unsigned short	无符号 短整型	%hu	2字节	0	
unsigned long	无符号 长整型	%lu	windows: 32/64位: 4字节 Linux下: 32 位: 4字节、64位: 8字节。	0	
unsigned long long	无符号 长长整 型	%llu	8字节	0	